Blood Bank Donor & Inventory Management System

Midterm Project Report — Web Technologies and API Services

Master's in Big Data Analytics

GROUP 2

GROUP MEMBERS

ID	Name
20251MBI012	Izabayo Isabelle
20251MBI050	Hirwa Fabrice
20251MBI014	Masengesho Donatien

1. Introduction

This project is a database-driven web application built with Django to support the day-to-day operations of a blood bank network: registering donors, running donation campaigns, recording and lab-testing collected blood units, and matching that stock against blood requests raised by hospitals. The application solves a coordination problem that is normally spread across spreadsheets and phone calls; knowing in one place how much blood of each group is actually available, screened and unexpired, and which requests are still outstanding against it.

The system is organised as three Django apps that mirror three real operational concerns: core (institutions and staff), donors (the donor registry, campaigns and the donation-to-blood-unit pipeline) and inventory (blood requests, issuance and stock thresholds). A shared AdminLTE 3 dashboard gives staff a single, authenticated entry point into all three.

1.1 Objectives

- Model a realistic, multi-entity blood bank domain in Django with meaningful foreign-key and one-to-one relationships, not a single flat table.
- Provide full CRUD coverage (create, read, update, delete) for the core records; Donors, donation campaigns, donations, blood requests through validated Django forms rather than raw database access.
- Enforce the domain's real business rules in code (donor eligibility, the 56-day donation interval, hemoglobin thresholds, expiry handling, stock matching) so the data in the system stays trustworthy, not just structurally valid.
- Integrate the AdminLTE 3 template properly into Django's template-inheritance system, with a live sidebar, summary cards and database-backed tables, not a static HTML page.
- Expose the same data through Django's built-in admin site, and gate every operational page behind authentication.

2. System Design

2.1 Application structure

The project is split into three Django apps, each owning one slice of the domain:

App	Responsibility	Key models
core	Institutions (hospitals / blood banks) and which staff user belongs to which institution; also hosts the shared dashboard and login views.	Institution, InstitutionMembership
donors	The donor registry, donation campaigns, individual donation	Donor, DonationCampaign,

	records and the blood units a collected donation produces.	Donation, BloodUnit
inventory	Blood requests raised by hospitals, the units issued against them, and per-blood-group minimum stock thresholds.	BloodRequest, IssuedUnit, StockThreshold

Table 1. The two required Django apps are exceeded here with three, each with a clear single responsibility.

2.2 Database design and relationships

The data model has ten entities, the nine application models above plus Django's built-in auth.User, which several models reference for accountability (who recorded a donation, who handled a request, who issued a unit).

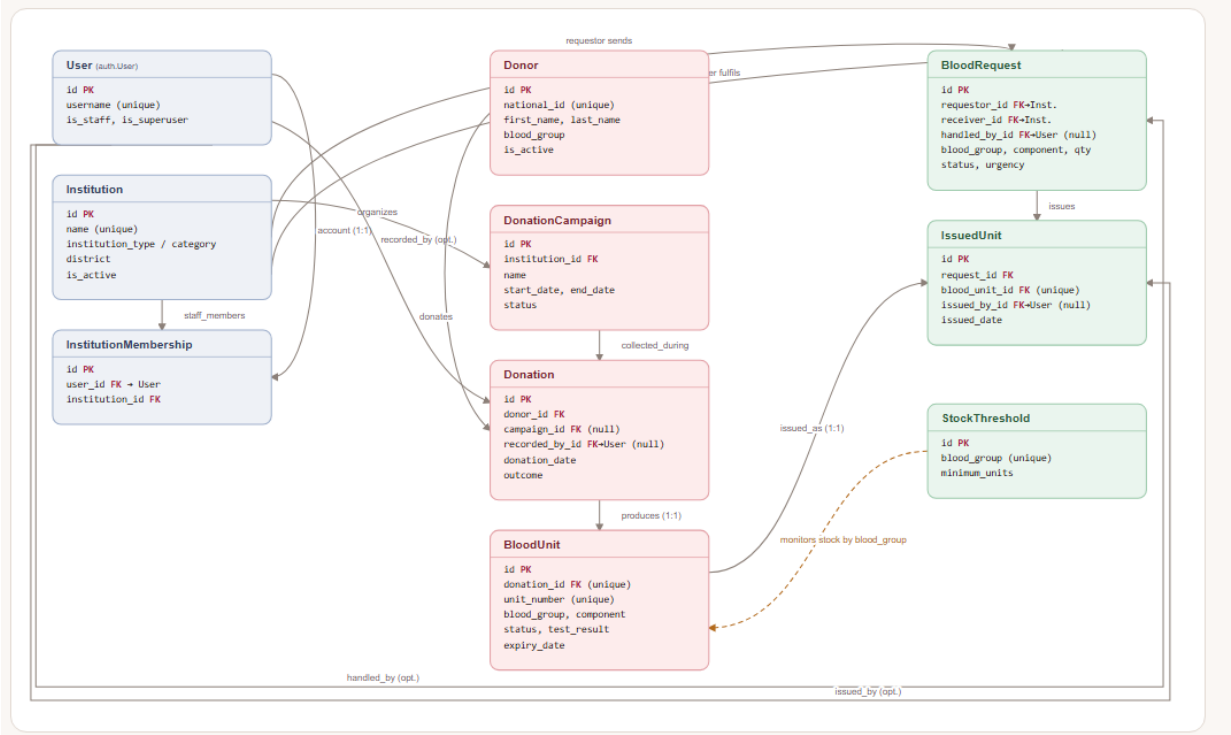


Figure 1. Entity-relationship diagram — core (blue), donors (red) and inventory (green) apps, plus Django's auth.User.

The design satisfies the brief's relationship requirement well beyond the minimum: it uses five ForeignKey relationships pointing at auth.User alone (recorded_by, handled_by, issued_by, plus the membership link), a self-referencing pair of foreign keys from BloodRequest to Institution (requestor and receiver — two independent relationships to the same table), and three OneToOneField relationships (InstitutionMembership↔User, BloodUnit↔Donation, IssuedUnit↔BloodUnit). The cardinality of every relationship is explained in the table below.

Relationship	Cardinality	Meaning
User → InstitutionMembership	1 : 1	A staff account has at most one

		institution membership (OneToOneField).
Institution → InstitutionMembership	1 : *	One institution has many staff members.
Institution → DonationCampaign	1 : *	An institution organises many campaigns over time.
Donor → Donation	1 : *	A donor can make many donations, subject to the 56-day eligibility rule enforced in Donor.is_eligible_on().
DonationCampaign → Donation	1 : * (optional)	A campaign can group many donations; a donation may also stand alone (null=True).
User → Donation	1 : * (optional)	The staff member who recorded the donation.
Donation → BloodUnit	1 : 1	A collected donation produces exactly one trackable unit (OneToOneField); a deferred donation produces none.
Institution → BloodRequest (requestor)	1 : *	The institution sending the request — usually a hospital.
Institution → BloodRequest (receiver)	1 : *	The institution asked to fulfil it — usually the blood bank. A second, independent FK to Institution.
User → BloodRequest	1 : * (optional)	The staff member who handled the request.
BloodRequest → IssuedUnit	1 : *	A request can be fulfilled by several individual issuances.
BloodUnit → IssuedUnit	1 : 1	A specific unit can be issued once (OneToOneField).
User → IssuedUnit	1 : * (optional)	The staff member who released the unit.
StockThreshold ↔ BloodUnit	logical only	Not a foreign key — compared at query time on blood_group to drive the dashboard's low-stock alerts.

Table 2. All fourteen relationships in the schema, with cardinality and intent.

3. Implementation

3.1 URL routing and views

Each app owns its own urls.py, included from the project-level blood_bank_donor/urls.py, demonstrating the required project-level / app-level split. The system implements well over the minimum five meaningful views — eighteen routes in total — several of them taking a dynamic <int:pk> URL parameter (donor_profile, donor_edit, campaign_edit, request_edit, issue_units) to resolve a specific database record from the URL itself.

URL	View	Purpose
/	DashboardView	Summary cards, low-stock alerts, stock chart, recent activity.
/accounts/login/	SystemLoginView	AdminLTE-styled login form.
/institutions/	InstitutionListView	List partner institutions.
/institutions/add/, /<pk>/edit/	InstitutionCreateView / UpdateView	Create / edit an institution.
/donors/donors/	DonorListView	Paginated donor registry.
/donors/donors/<pk>/	DonorDetailView	Donor profile — dynamic URL parameter.
/donors/donors/add/, /<pk>/edit/	DonorCreateView / UpdateView	Create / edit a donor.
/donors/campaigns/, /add/, /<pk>/edit/	CampaignListView / CreateView / UpdateView	Manage donation campaigns.
/donors/donations/, /add/	DonationListView / CreateView	Record and list donations.
/donors/donations/<pk>/delete/	delete_donation_session	Remove a donation record.
/inventory/.../requests/, /add/, /<pk>/edit/	BloodRequestListView / CreateView / UpdateView	Manage hospital blood requests.
/inventory/.../requests/<pk>/issue/	IssueUnitsView	Match and issue stock against one request.
/inventory/.../requests/<pk>/process/	process_issuance	Handle the issuance form POST.
/inventory/.../issued/<pk>/remove/	remove_issued_unit	Undo an issuance, returning stock to AVAILABLE.

Table 3. Representative routes. requestor/receiver on BloodRequest and the ForeignKey chain show how a URL parameter resolves through get_object_or_404 to a model instance and back out to a rendered template.

Known quirk: donors.urls and inventory.urls are both included under a path prefix that repeats their own first segment (path("donors/", include("donors.urls")) where donors/urls.py already starts its own patterns with donors/), so the live paths are /donors/donors/... and /inventory/inventory/.... Functionally everything still resolves correctly end-to-end — it is simply not the shortest possible URL — and is noted here as a known limitation rather than silently left unexplained.

3.2 Templates, template inheritance and AdminLTE integration

All authenticated pages extend a single templates/adminlte/base.html, which carries the AdminLTE 3 shell, sidebar navigation, top navbar, preloader and the static-asset includes, and exposes a {% block content %} that every page template fills in. This is template inheritance in the Django sense, not a static template copy-pasted onto each page: static files (CSS, JS, fonts, images) are served through {% static %} and STATICFILES_DIRS, and the AdminLTE source tree itself lives under static/, separate from the Django page templates under templates/adminlte/.

The dashboard (templates/adminlte/dashboard.html) is the clearest example of live integration: its four summary cards, the low-stock alert banner, the Chart.js stock-by-blood-group bar chart and the recent-requests table are all rendered from context data computed in DashboardView.get_context_data() — none of it is hard-coded markup.

3.3 CRUD operations and validation

Every primary entity (Donor, DonationCampaign, Donation, Institution, BloodRequest) has create and update views built on Django's ModelForm-backed CreateView / UpdateView, list views for read, and a detail view for the donor profile. Deletion is implemented for donations and for individual issued units (remove_issued_unit), the latter deliberately reversible — removing an issuance returns the unit to AVAILABLE stock rather than destroying history.

Validation is enforced at the model layer via clean() and full_clean(), so it applies whether a record is created through a form, the admin site, or a management command. Representative rules:

- Donor: age must be 18–65 and weight ≥ 50 kg at save time.
- Donation: hemoglobin < 12.5 g/dL forces outcome=DEFERRED with a reason; a collected donation re-checks the donor's 56-day eligibility window and campaign date range.
- BloodUnit: expiry date is derived automatically from the component's shelf life (35 days whole blood, 42 red cells, 365 plasma, 5 platelets) and a REACTIVE lab result can never be AVAILABLE.
- BloodRequest / IssuedUnit: a unit can only be issued if its blood group and component match the request exactly, if it is unexpired and lab-cleared, and if the request has not already received its full quantity.

3.4 Django admin and authentication

All nine application models are registered in their app's admin.py with tailored list_display, list_filter and search_fields (for example BloodRequestAdmin nests its IssuedUnit records inline, and StockThresholdAdmin surfaces the computed current_stock and is_low_stock properties directly in the change list). A superuser

account was created with python manage.py createsuperuser and used to demonstrate record management through /admin/ end-to-end.

Every operational page (dashboard, donor/campaign/donation/institution/request views) is restricted to authenticated users via LoginRequiredMixin or @login_required. Login and logout are implemented with Django's own auth views (SystemLoginView extends LoginView; LogoutView is used directly), rendering through a bespoke AdminLTE-styled template rather than Django's default form. LOGIN_URL, LOGIN_REDIRECT_URL and LOGOUT_REDIRECT_URL are set explicitly so an unauthenticated visit redirects to the login page and a successful login lands back on the dashboard.

4. Screenshots

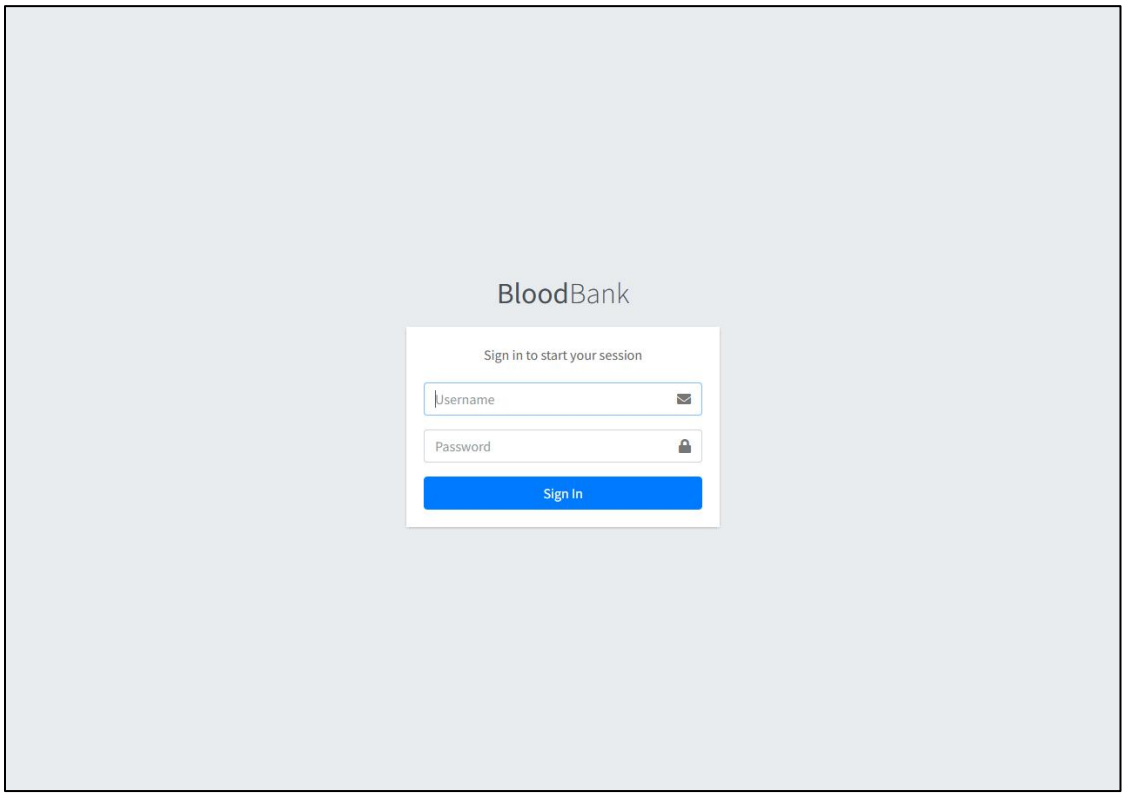


Figure 2. AdminLTE-styled login page (SystemLoginView).

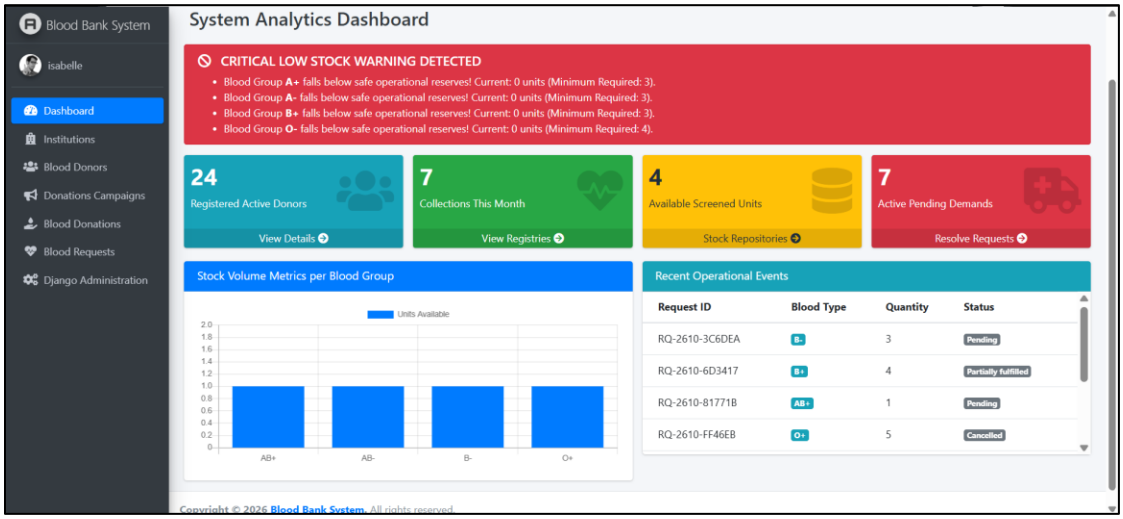


Figure 3. Dashboard — summary cards, low-stock alerts and stock chart.

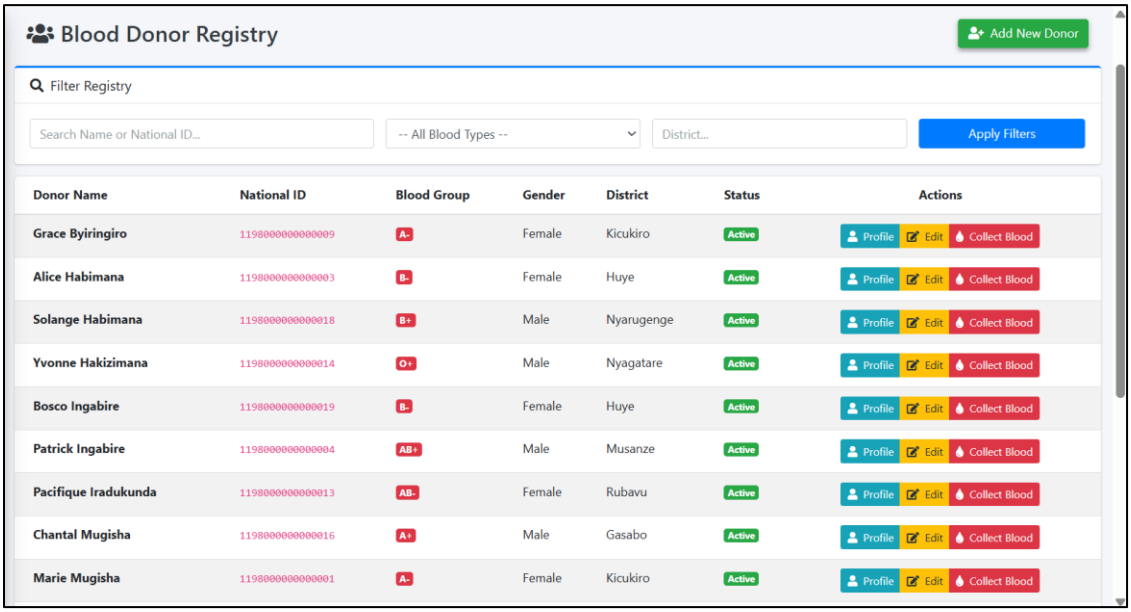


Figure 4. Donor list (paginated, database-backed table).

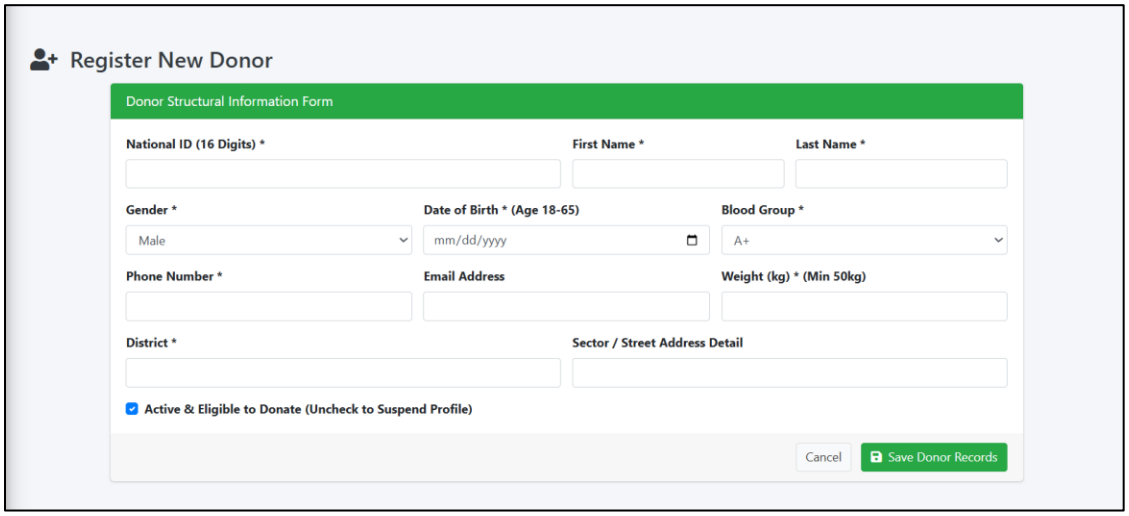


Figure 5. Add Donor form (ModelForm with validation).

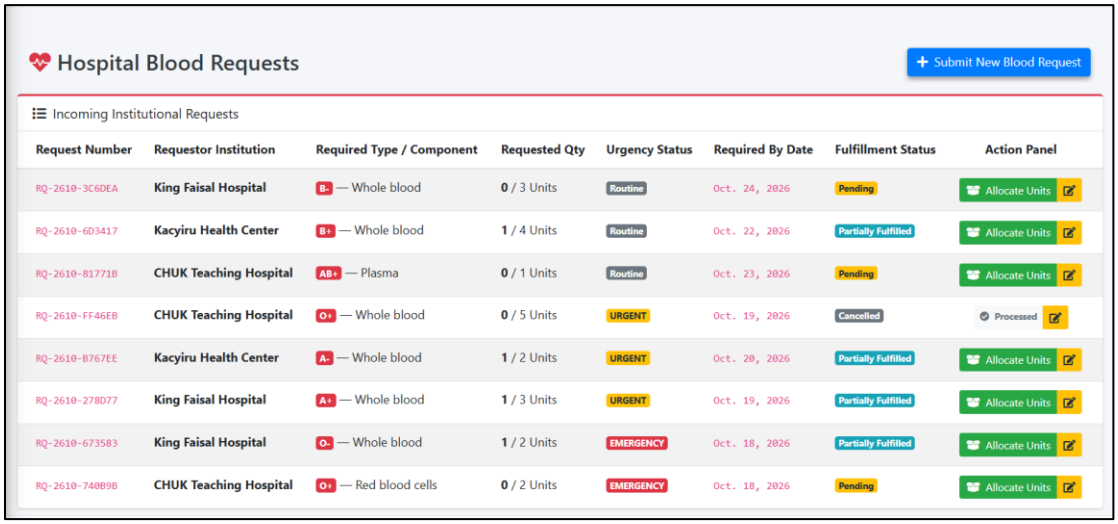


Figure 6. Blood request list across institutions.

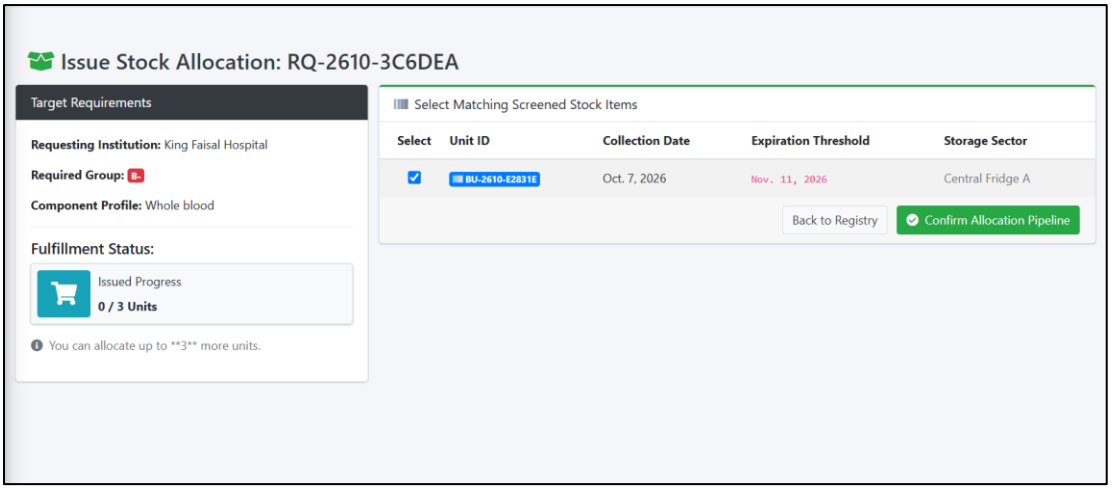


Figure 7. Issue Units — matching available stock against a request.

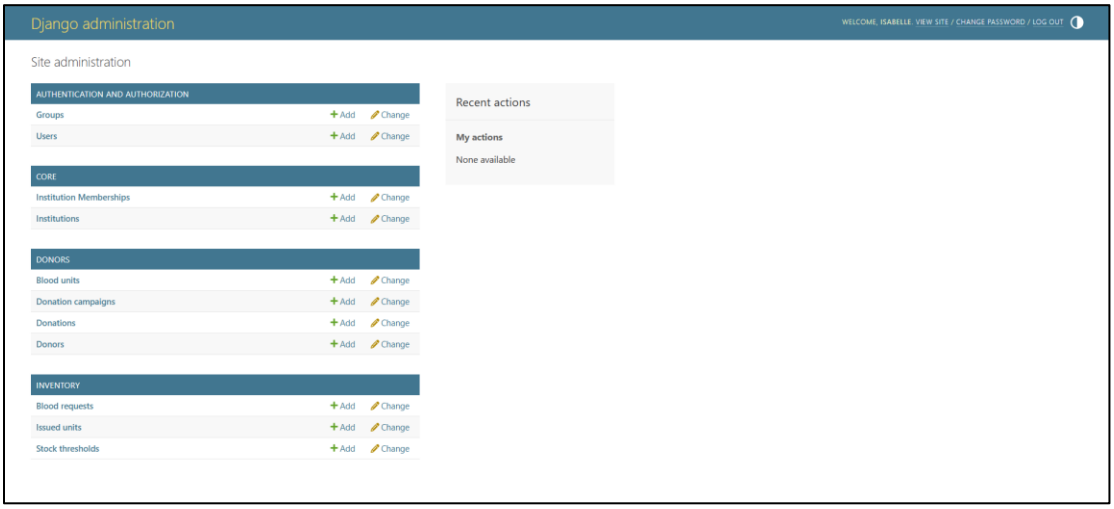


Figure 8. Django admin site, listing all registered models.

5. Challenges and how they were resolved

Most of the difficulty in this project was not writing Django code in isolation, but the ordinary things a student group runs into the first time it builds something this size together: coordinating schema changes across three people, learning that a downloaded admin template is not automatically a working Django app, and discovering that “it matches the field types” is not the same as “it matches the real-world rule”.

Coordinating schema changes as a group

With three people committing to the same models over several pull requests, a model (BloodUnit) ended up being moved from the inventory app to the donors app partway through, and the new migration this produced reused the exact same database index names as the old one (e.g. unit_group_status_idx). Index names are database-global in SQLite, so a teammate running migrate on a fresh clone hit "index unit_group_status_idx already exists". This was a useful early lesson in treating migrations as shared, ordered history rather than private scratch files: whoever restructures a model needs to check for naming clashes against every app, not just their own. Fixed by giving the donors-side indexes a distinct donor_unit_*_idx prefix.

Django's own defaults not matching the project's URLs

A related surprise: once login worked, a successful sign-in redirected to Django's built-in default, /accounts/profile/, which this project never defines, producing a 404 immediately after a correct login. It is an easy thing to miss because the login form itself looks completely fine right up until that last step. Fixed by setting LOGIN_URL, LOGIN_REDIRECT_URL and LOGOUT_REDIRECT_URL explicitly in settings.py.

Translating real-world rules into validation, not just field types

A CharField or a DecimalField will happily accept a donor who is 12 years old or a donation recorded for next week; Django's field types only constrain shape, not meaning. The harder, more instructive part was encoding the domain's actual rules (donor age 18–65, the 56-day gap between donations, hemoglobin-based deferral, expiry derived from the component's real shelf life, exact blood-group/component matching before

a unit can be issued) as explicit clean() logic routed through full_clean() on save, so the same rule applies everywhere — form, admin site or script — instead of being re-implemented, and possibly forgotten, in each view.

6. Conclusion

The project meets every minimum technical requirement in the brief — three Django apps rather than two, eighteen routed views with dynamic URL parameters, a ten-entity schema with fourteen foreign-key / one-to-one relationships explained above, full CRUD with model-level validation, a working AdminLTE dashboard fed entirely by live queries, and Django admin plus authenticated, session-based access control, while staying grounded in a real operational problem: knowing what blood is actually in stock and matching it safely against demand.

7. Appendix: Demo data

A custom management command, core/management/commands/seed_demo.py, populates the database with a realistic working set so the dashboard, charts and lists have meaningful content instead of an empty shell: 4 institutions, 24 donors spread across all 8 blood groups, 2 donation campaigns (one ongoing, one completed), 24 donations (including a few hemoglobin-deferred ones), the blood units they produce in a mix of lab states (cleared/available, pending, reactive/discarded, expired), 8 blood group stock thresholds, and 8 blood requests of varying urgency and status with several units already issued against them. It is idempotent by default (python manage.py seed_demo) and supports a full reset (python manage.py seed_demo --reset).

8. References

[1] Kundan Kumar. Django Web Development — Midterm Examination: Project Work (course assignment brief). Web Technologies and API Services, AUCA, 2026.

[2] Django Software Foundation. Django — Getting Started. https://www.djangoproject.com/start/

[3] Django Software Foundation. Django Documentation. https://docs.djangoproject.com/en/stable/

[4] ColorlibHQ. AdminLTE 3 Documentation. https://adminlte.io/docs/

[4] W3Schools. Django Tutorial. https://www.w3schools.com/django/

[5] Kundan Kumar. VS Code for Python and Django Setup Guide | Install and Configure Like a Pro (video, lecturer's own channel).